03 MAR 2026

AADK TASK 2

Responsive List & Image Loading Strategy

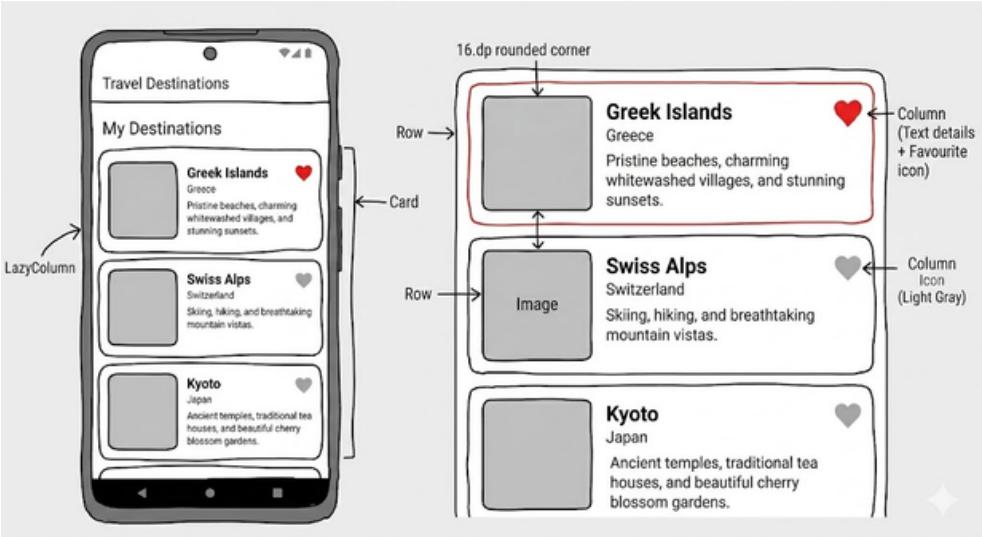
MindMatrix

1. Responsive Behavior Plan

Theme: Travel Destinations (Continuation from Task 1)
In this task, I planned how my travel destinations list will behave on different screen sizes and how images will be loaded efficiently without affecting performance.

Mobile (Portrait Mode)

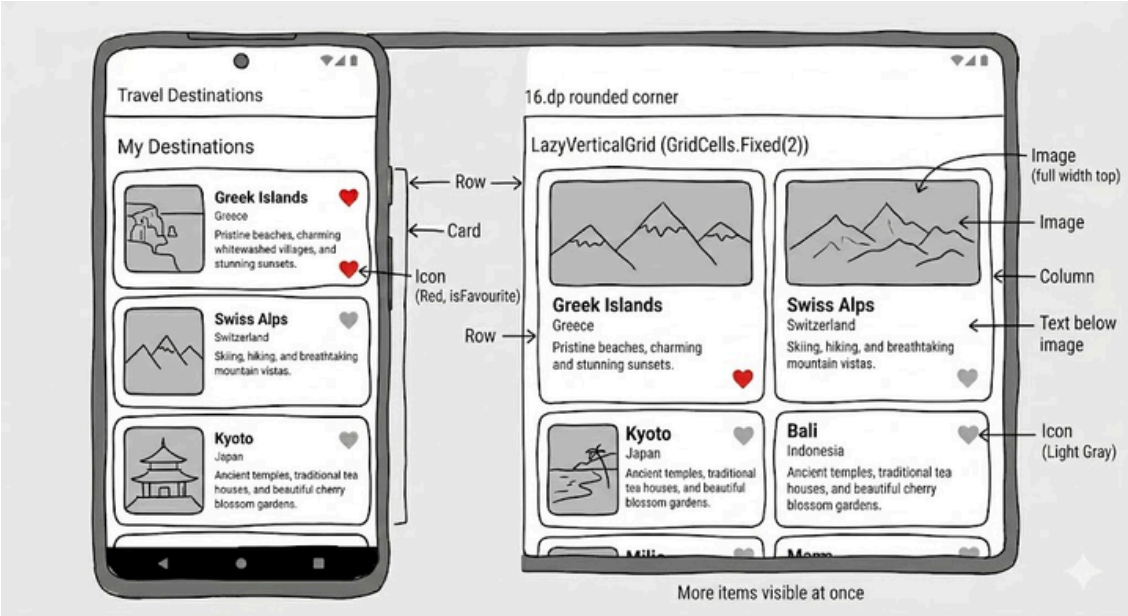
- Layout: Single Column List
- Component: LazyColumn
- Each item takes full width of screen.
- Image on left, text on right inside a Card.



This layout is simple and easy to scroll on phone screens.

Tablet / Landscape Mode

- Layout: Two-Column Grid
- Component: LazyVerticalGrid
- Each item becomes slightly smaller and arranged side-by-side.



In tablet mode:

- Image will be placed on top (full width inside card).
- Text will appear below image.
- More items visible at once.

Why I chose LazyColumn & LazyVerticalGrid?

- LazyColumn is best for vertical scrolling lists in phones.
- LazyVerticalGrid helps use extra space efficiently in tablets.
- Both are “lazy” components, meaning they only load visible items, which improves performance.

2. Image Loading Strategy

Image Source Decision

For this project, I plan to use:

- Internet image URLs
- Loaded using Coil library with Jetpack Compose

Example approach:

```
AsyncImage(
    model = imageUrl,
    contentDescription = title
)
```

Why I chose Coil?

- Officially recommended for Compose.
- Easy integration.
- Handles caching automatically.
- Good performance and memory management.

Placeholder Strategy

While image is loading:

- Show a light gray box or loading indicator.
- Use placeholder image (like R.drawable.placeholder).

If image loading fails:

- Show an error image (broken image icon).
- Prevent app crash by handling failure state.

Example plan:

- placeholder = R.drawable.placeholder
- error = R.drawable.error_image

Performance Tips I Will Follow

- Use properly sized images (not very high resolution).
- Avoid loading full 4K images in list.
- Use Coil caching to avoid re-downloading images.
- Use Lazy components to avoid loading off-screen items.
- Compress images before adding to drawable folder.

3. Resource & Memory Considerations

Target Image Dimensions

For list items:

- Around 200dp × 200dp
- Resolution approx: 800×800 px max (compressed)

For grid view:

- Slightly smaller images to fit two columns.

Managing Large Lists

If list becomes large (50+ items):

- Use Pagination (Paging 3 library).
- Load data in small batches.
- Avoid storing huge image files locally.
- Use lazy loading so only visible items consume memory.

Memory Safety Plan

- Avoid using large bitmap files.
- Use vector images when possible.
- Rely on Coil caching instead of manually storing images.
- Clear unused resources properly.

4. Reflection

Image-rich scrollable lists can cause performance problems like lagging and high memory usage if not handled properly. Large images take more memory and may slow scrolling. Responsive layout makes it more complex because we must adjust item size and arrangement based on screen width. If not planned properly, UI can look stretched or messy on tablets. Performance matters a lot in mobile apps because users expect smooth scrolling and fast loading. Proper image handling and responsive design improve user experience and make the app feel professional.